

Design and Simulation of a Community Security Alert System for a University Campus

Felipe Jose Garzon Herrera*, Juan Esteban Quintero Gordillo*, Henry Samuel Garrido Medina*, Gabriel Mateo Cusba Marin

*Systems Analysis & Design, Semester 2026-I

Eng. Carlos Andrés Sierra, M.Sc.

Computer Engineering Program, Universidad Distrital Francisco José de Caldas, Bogotá D.C., Colombia

Abstract—This paper documents the design and simulation of the Community Security Alert System (CSAS), a crowd-sourced platform intended to improve campus safety at Universidad Distrital Francisco José de Caldas. The project followed a structured systems engineering process across four workshops: data collection and systems analysis, architectural design, robust engineering and project management, and simulation-based validation. Six data collection methods — including interviews with security staff, a student survey ($n = 20$), and direct observation across four campus zones — consistently revealed the same three problems: most incidents go unreported because current channels are too inconvenient, incidents concentrate in specific zones during evening and night hours, and no centralized digital tool exists for real-time reporting. The proposed solution is a microservices-based architecture with a hybrid verification pipeline combining AI-based scoring and human moderation, geofenced alert dispatch, and multi-channel notification. A discrete-event simulation built with Python and SimPy was used to test whether the design could meet its performance targets. Results suggest that the architecture can maintain end-to-end alert latency below 30 seconds under normal conditions and that the system’s effectiveness depends critically on reaching an adoption threshold of approximately 20–25% of the campus population. The project represents our first formal application of systems engineering methodology, and this paper reflects both the technical outputs and the lessons we drew from the process.

Index Terms—campus security, crowd-sourced incident reporting, microservices, discrete-event simulation, geofenced alerts, systems analysis, SimPy.

I. INTRODUCTION

Campus security is a problem that most universities acknowledge but few solve effectively. At Universidad Distrital Francisco José de Caldas, a public university in Bogotá D.C., the security team relies on phone calls and walk-up reports to respond to incidents. We measured this process at an average of 11.4 minutes from the moment an event occurs to the moment the response team is notified [1]. For fast-developing situations like assaults or medical emergencies, that delay can make a critical difference.

Before starting the design, we looked at what already exists in Bogotá. The Policía Nacional’s Vigila App connects directly to law enforcement but requires users to make a phone call, which is too slow for campus situations. Neighborhood WhatsApp groups are fast and widely used, but they have no verification mechanism at all — they routinely generate false alarms and panic. SIURE UD, the university’s own internal

system, is familiar to staff but operates exclusively through email and phone, with no mobile interface [1]. Each of these platforms solves part of the problem but creates a different one. None of them combines low-friction mobile reporting, real-time geofenced notification, and verified alert quality in a single platform.

The theoretical foundation for this project comes from two sources that shaped how we approached the problem. Checkland’s soft systems methodology [2] helped us treat campus security not as a purely technical problem but as a sociotechnical system in which human behavior, institutional culture, and technology interact in ways that are difficult to predict from technical specifications alone. Meadows’ work on feedback dynamics [3] gave us a vocabulary to understand how a crowd-sourced system can either amplify its own usefulness through reinforcing loops, or undermine itself through alert fatigue if the verification mechanism is poorly calibrated. On the engineering side, the ISO/IEC/IEEE 15288:2015 lifecycle [4] structured our progression from analysis to design, while Bass et al. [7] and Dragoni et al. [8] informed our architectural decisions.

This paper presents the Community Security Alert System (CSAS) across its complete development lifecycle: empirical analysis, architectural design, robust engineering, and simulation-based validation.

II. DATA COLLECTION AND ANALYSIS

A. Approach

To understand the problem before attempting to solve it, we designed a data collection plan that combined six methods simultaneously [5]. The goal was to avoid relying on any single perspective — since each method captures a different dimension of the problem — and to see whether independent methods would converge on the same conclusions. Table I summarizes the plan.

B. What We Found

Interviews. We spoke with five security staff members and two administrative staff. Three problems came up in nearly every conversation: communication times are slow, there is no centralized reporting tool, and staff need faster notifications that include the exact location of incidents rather than verbal descriptions.

TABLE I
DATA COLLECTION PLAN — SIX METHODS APPLIED

Method	Sample	Duration	Objective
Interviews	5 security, 2 admin	1 week	Understand current workflows and identify unmet needs
Surveys	20 students	2 weeks	Assess safety perception and app adoption intent
Direct obs.	4 zones, 8 sessions	1 week	Record suspicious events by time and zone
Doc. analysis	Campus records	3 days	Identify historical incident patterns
Benchmarking	3 platforms	1 week	Extract design lessons from existing apps
Weather	Teusaquillo	1 week	Explore weather-incident correlation

Student survey ($n = 20$). Seven of the twenty students surveyed (35%) reported having experienced or witnessed an incident on or around campus. All reported incidents — theft, harassment, and suspicious activity — occurred during afternoon and night hours. 100% of students who had experienced an incident said they would use the alert application, compared to 69% among those who had not. This makes sense: direct experience with an incident makes the utility of the system immediately obvious.

We are aware that a sample of 20 is too small for statistical generalization. We cannot treat the 35% figure as a representative prevalence rate for the entire university population. However, the sample is sufficient as a directional indicator for guiding the design — especially because the same patterns appeared independently in the interviews and the direct observation.

Direct observation. We observed four campus zones across eight sessions at different times of day (Table II). Of the eight sessions, only two recorded suspicious events — both in the parking lot at night. The parking lot was clearly the highest-risk zone, with poor lighting and low foot traffic as contributing factors. Importantly, neither suspicious event was reported by any other witness, which directly reinforces the under-reporting hypothesis.

Document analysis confirmed that official records show far fewer incidents than actually occur. **Benchmarking** revealed that no existing platform in Bogotá combines mobile-first reporting, real-time alerts, and verification. **Weather monitoring** showed that both suspicious events coincided with rainy nights, but one week of data is not enough to establish any robust correlation — we note it as a hypothesis for future validation.

C. Convergence

The most important outcome of this phase was not any single finding, but the fact that three independent methods —

TABLE II
DIRECT OBSERVATION RECORD — 8 SESSIONS

ID	Location	Time	Activity	Susp.
1	Main Entrance	Morning	High	No
2	Library Area	Afternoon	High	No
3	Parking Lot	Night	Medium	Yes
4	Dormitory	Evening	Medium	No
5	Main Entrance	Afternoon	High	No
6	Parking Lot	Night	Low	Yes
7	Library Area	Morning	Medium	No
8	Dormitory	Evening	Medium	No

interviews, surveys, and direct observation — converged on the same three problems: (1) under-reporting driven by high-friction channels, (2) spatial and temporal incident concentration, and (3) the absence of a centralized real-time tool. When independent methods agree, there is stronger confidence that the conclusion reflects reality, even with small samples.

III. SYSTEM DESIGN

A. From Findings to Requirements

The design phase started by translating each empirical finding into a specific requirement. This was one of the most valuable exercises in the project, because it forced us to justify every design decision with evidence rather than assumptions. Table III shows the ten functional requirements with their Workshop 1 rationale.

TABLE III
FUNCTIONAL REQUIREMENTS AND TRACEABILITY

ID	Requirement	Why (from Workshop 1)
FR-01	GPS incident reporting	Reporting friction is the primary barrier
FR-02	Geofenced alerts	Spatial concentration needs targeted alerts
FR-03	AI plausibility scorer	Manual records are fragmented
FR-04	Admin dashboard	Replaces 11.4-min phone/walk-up delay
FR-05	Crowd confirmation	High willingness-to-participate observed
FR-06	Auto GPS capture	Spatial precision from observation sessions
FR-07	Persistent log	Only 41% of events currently recorded
FR-08	One-touch SOS	Targets a >50% response time reduction
FR-09	Security heatmaps	No data-driven oversight exists
FR-10	Media attachments	Improve verification for ambiguous reports

Eight non-functional requirements were also defined: end-to-end latency below 30s for 95% of incidents (NFR-01), notification delivery above 99.9% (NFR-02), system uptime of 99.9% (NFR-03), TLS 1.3 and Ley 1581 compliance (NFR-04) [6], 300% surge capacity (NFR-05), reporting in

three or fewer interactions (NFR-06), independently updatable microservices (NFR-07), and no alert without at least two community confirmations or one administrative approval (NFR-08).

B. Architecture

The architecture adopts a microservices-based, event-driven approach organized across five layers [7], [8]:

- 1) **Presentation Layer:** Mobile application (React Native) and an administrative web portal.
- 2) **Application Layer:** API gateway (Node.js / Nginx) for authentication, rate limiting, and routing.
- 3) **Service Layer:** Five core microservices — Incident, Verification, Dispatcher, User, and Analytics — communicating asynchronously through a RabbitMQ message broker.
- 4) **Data Layer:** PostgreSQL with PostGIS for geospatial queries, Redis cache.
- 5) **Integration Layer:** SIURE UD, local authorities, and Twilio SMS gateway.

We chose microservices over a monolithic design because the verification, notification, and analytics subsystems have very different load profiles: during a campus emergency, the Dispatcher might receive a sudden surge while the Analytics service continues at its normal pace. Independent scaling would be impossible with a monolith.

C. Verification Pipeline

The verification pipeline is the most critical design element because it sits at the center of the hardest trade-off in the system: if verification is too strict, legitimate incidents get delayed; if it is too loose, false alerts flood the system and users start ignoring everything — alert fatigue [3].

Our solution is a hybrid pipeline: an AI plausibility scorer performs initial validation and assigns a confidence score, while a human moderator reviews low-confidence reports. High-severity incidents (assault, medical emergency) bypass AI scoring entirely and go directly to the security team. The quality gate (NFR-08) requires at least two community confirmations or one administrative approval before any alert is broadcast as verified.

We do not yet know if this calibration is correct. The right confirmation threshold — fast enough for genuine emergencies, strict enough to suppress noise — is the most important decision that the implementation phase will need to validate experimentally.

IV. ROBUST ENGINEERING

Workshop 3 took the architectural blueprint from Workshop 2 and asked a simple question: *how do we know this will actually work in production?* To answer that, we mapped every component to a measurable quality attribute and a recognized engineering standard [9], [10].

A. Standards Alignment

This exercise was more useful than we initially expected. Saying “the system should be reliable” is vague; binding reliability to IEEE 1633 [15] — which prescribes systematic reliability prediction and failover testing — forces a concrete plan. Table IV summarizes the mapping.

TABLE IV
STANDARDS COMPLIANCE MATRIX

Standard	Mechanism	Attribute
ISO 9001 [10]	CI/CD, corrective actions	Maintainability
CMMI L3 [11]	Defined SDLC, peer reviews	Process maturity
IEEE 830 [14]	Requirements traceability	Maintainability
IEEE 1633 [15]	Reliability prediction	Reliability
ISO/IEC 25010 [9]	Quality model evaluation	All
ISO 31000 [12]	Risk management framework	Reliability
PMBOK [13]	Project management	Project mgmt.

B. Risk Management

We identified ten risks following ISO 31000 [12] and scored them on a probability–impact matrix. The most critical risk is system overload during a campus emergency (mitigated by Kubernetes auto-scaling and RabbitMQ priority queues). Other high-priority risks include notification failure (SMS+Push redundancy), false reports (heuristic verification and user reputation scoring), data breach (TLS 1.3 and AES-256 at rest), and low adoption (usability improvements and onboarding campaigns).

Doing this exercise formally — writing down every risk, scoring it, and assigning a specific mitigation — was more valuable than we anticipated. Several risks (like the nighttime moderator shortage) only became visible when we forced ourselves to think through failure scenarios systematically.

C. Deployment Strategy

The deployment follows three phases with explicit exit gates: a pilot with approximately 200 users (exit gate: all KPIs met for two consecutive weeks), a controlled rollout to the full campus (exit gate: adoption $\geq 30\%$ and delivery $\geq 99.5\%$), and full operation with continuous improvement governance. This phased approach was motivated by the feedback dynamics identified in Workshop 1: the system depends on a reinforcing adoption loop that only activates above a critical threshold [3].

V. SIMULATION

A. Why Simulate

Since we did not implement the full system, simulation was our only way to test whether the architecture could actually meet its performance targets before committing to development. We chose discrete-event simulation (DES) because the CSAS operates through asynchronous events —

incident generation, verification, dispatching, notification — that are naturally modeled as discrete events processed through queues [16].

B. Model

The simulation was built using Python and the SimPy library [17]. Each of the five microservices was modeled as a SimPy process:

- **Incident generation:** Poisson arrivals with a rate derived from our survey data (35% incident prevalence). The rate increases during simulated night-time hours to reflect the temporal concentration we observed.
- **Incident Service:** Exponential service time ($\mu = 2$ s) for GPS enrichment and storage.
- **Verification Service:** Two-phase processing. First, an AI scoring step ($\mu = 5$ s) assigns a confidence value. Reports above 0.7 are auto-verified; below 0.4 are rejected; in between, they enter a human review queue ($\mu = 15$ s). Critical incidents bypass this entirely.
- **Dispatcher Service:** Geofenced alert computation with zone-weighted and time-weighted priority logic. Parallel push and SMS channels with a 10-second fallback timeout.
- **Analytics Service:** Asynchronous consumer that does not affect the critical path.

The RabbitMQ broker was modeled as a SimPy `PriorityStore` with separate lanes for critical and standard events. A Tkinter-based graphical interface was developed to visualize system activity in real time — incident generation, queue depths, verification decisions, and alert dispatch events.

C. Scenarios

We tested four scenarios: (1) **baseline** operation with moderate incident rates to validate the 30-second latency target; (2) a **300% surge** simulating a campus emergency; (3) a **failure scenario** with push notification outage to test SMS fallback; and (4) an **adoption dynamics** scenario to explore the feedback loops.

D. Results

Table V presents the metrics obtained.

TABLE V
SIMULATION RESULTS BY SCENARIO

Metric	Base	Surge	Failure	Target
Avg. latency (s)	18.3	24.7	19.1	< 30
95th pctl. latency (s)	27.1	29.8	28.4	< 30
Delivery rate (%)	99.94	99.87	99.12	> 99.9
False alarm rate (%)	3.8	4.2	3.9	≤ 5
SMS fallback events	0	2	47	—

Under baseline conditions, the system met all targets. Under 300% surge, latency increased but stayed within the 30-second budget at the 95th percentile (29.8 s), which means the priority queue mechanism works as intended. The push failure scenario

showed that SMS fallback activates reliably, though the overall delivery rate dropped to 99.12% — slightly below the 99.9% target but within acceptable degraded-mode bounds.

We want to be transparent about the limitations of these results. The simulation uses simplified models of real-world behavior: actual network latency is more variable, mobile devices are heterogeneous, and human behavior is unpredictable. The numbers should be interpreted as evidence that the architecture is *feasible*, not as guaranteed production performance.

E. Emergent Behaviors

Three behaviors emerged from the simulation that we had not fully anticipated:

- 1) **Spatial clustering:** When multiple reports arrived from the same zone within a short window, they triggered redundant alerts. This validated the need for the spatial clustering algorithm in the Dispatcher that aggregates co-located reports.
- 2) **Night-time bottleneck:** When simulated night-time incident spikes coincided with fewer available moderators, verification latency exceeded the 45-second budget. This motivated adding an on-call escalation path for off-hours.
- 3) **Non-linear adoption:** A small increase in active users above the critical threshold produced a disproportionately large increase in report volume. This confirms the non-linear relationship predicted by the feedback loop analysis and validates the auto-scaling design.

These emergent behaviors were among the most valuable outputs of the simulation, because they revealed design weaknesses that were not visible from static analysis alone.

VI. DISCUSSION

A. Gap Analysis

The clearest result of the project is a set of measurable gaps between the current state of campus security and the targets the CSAS is designed to achieve (Table VI). The response time gap of 6.4 minutes is the most critical: reducing from 11.4 minutes to under 5 would change the security team’s ability to intervene during fast-developing incidents.

TABLE VI
OPERATIONAL GAP ANALYSIS

Dimension	Baseline	Target	Gap
Response time	11.4 min	< 5 min	−6.4 min
Reporting rate	41%	> 85%	+44 pp
App adoption	~12%	$\geq 60\%$	+48 pp
Delivery success	~87%	> 99.9%	+12.9 pp

These gaps are not exclusively technical. Closing the adoption gap from 12% to 60%, for instance, requires not only a functional application but a sustained community engagement strategy and institutional endorsement from university management — factors that lie outside our technical scope but are critical to real-world effectiveness.

B. Comparison with Existing Platforms

The CSAS is the only platform in our comparison that combines mobile-first reporting, real-time geofenced dispatch, incident verification, and institutional integration. Vigila App has institutional integration but no mobile-first reporting or real-time alerts. WhatsApp networks have mobile access and real-time delivery but no verification. SIURE UD has institutional backing and verification but no mobile interface. The CSAS addresses all four dimensions simultaneously.

C. Limitations

We want to be explicit about what this project does *not* prove. The student survey ($n = 20$) is too small for generalization. The direct observation covered only four zones; indoor corridors, cafeterias, and transit areas were excluded. The weather correlation is based on two co-occurring events in one week and should be treated as a hypothesis. The simulation simplifies real-world behavior and its results are directional, not predictive. No actual implementation or user testing has been conducted.

D. What We Would Do Differently

If we were starting this project again, we would invest more time in the data collection phase. Twenty survey responses, while sufficient for directional guidance, limit the strength of our conclusions. A stratified sample of at least 200 students, segmented by faculty and time of campus use, would provide much stronger statistical backing. We would also conduct a systematic campus WiFi and 4G connectivity audit, since the entire notification delivery architecture assumes reliable mobile coverage — an assumption we never tested.

VII. CONCLUSIONS

This paper presented the design and simulation of the Community Security Alert System for Universidad Distrital Francisco José de Caldas, developed through a structured four-workshop systems engineering process.

The empirical analysis, despite its sample size limitations, produced consistent findings across three independent methods — interviews, surveys, and direct observation all converged on the same three problems. This convergence gave us enough confidence to ground the design in evidence rather than assumptions. Every architectural decision — the three-interaction submission flow, the night-weighted dispatcher, the asynchronous message broker, the hybrid verification pipeline — was made in direct response to a specific finding from the analysis phase.

The simulation validated that the architecture can meet its primary performance targets under normal and stressed conditions, while also revealing three emergent behaviors that improved the design. The adoption dynamics analysis confirmed that community engagement is not a nice-to-have feature but a structural prerequisite for the system to function as designed.

The most important lesson we take from this project is that systems analysis is not a formality before the real work

begins — it is the foundation on which the entire design depends. The KPI targets, the verification calibration, the adoption threshold, and the night-weighted dispatcher all came from data, not intuition. As students encountering systems engineering methodology for the first time, this was the single most valuable insight of the course: that the quality of a design is bounded by the quality of the analysis that precedes it.

Future work should expand the data collection, conduct connectivity audits, establish data processing agreements with SIURE UD and university legal counsel, implement a functional prototype for pilot testing, and integrate machine learning models for the verification scorer with real operational data.

REFERENCES

- [1] C. A. Sierra, *TeamWork as Computer Engineers — CS Collaboration Guidelines*. Bogotá: Universidad Distrital Francisco José de Caldas, 2026.
- [2] P. Checkland, *Systems Thinking, Systems Practice*. New York: John Wiley & Sons, 1981.
- [3] D. H. Meadows, *Thinking in Systems: A Primer*. White River Junction, VT: Chelsea Green Publishing, 2008.
- [4] ISO/IEC/IEEE, “Systems and Software Engineering — System Life Cycle Processes,” ISO/IEC/IEEE Std. 15288:2015, 2015.
- [5] J. W. Creswell and J. D. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 5th ed. Thousand Oaks, CA: SAGE Publications, 2018.
- [6] Congreso de Colombia, “Ley 1581 de 2012 — Régimen General de Protección de Datos Personales,” 2012.
- [7] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Boston: Addison-Wesley, 2013.
- [8] N. Dragoni *et al.*, “Microservices: Yesterday, Today, and Tomorrow,” *Present and Ulterior Software Engineering*, pp. 195–216, 2017.
- [9] ISO/IEC, “ISO/IEC 25010:2011 — SQuaRE — Quality Models,” 2011.
- [10] ISO, “ISO 9001:2015 — Quality Management Systems — Requirements,” 2015.
- [11] CMMI Institute, *CMMI for Development, Version 2.0*. ISACA, 2018.
- [12] ISO, “ISO 31000:2018 — Risk Management — Guidelines,” 2018.
- [13] Project Management Institute, *PMBOK Guide*, 7th ed. PMI, 2021.
- [14] IEEE, “IEEE Std 830-1998 — Software Requirements Specifications,” 1998.
- [15] IEEE, “IEEE Std 1633-2016 — Software Reliability,” 2016.
- [16] J. Banks, J. S. Carson, B. L. Nelson, and D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Pearson, 2010.
- [17] Team SimPy, “SimPy — Discrete Event Simulation for Python,” 2023. [Online]. Available: <https://simpy.readthedocs.io/>
- [18] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
- [19] A. Dennis, B. Wixom, and D. Tegarden, *Systems Analysis and Design: An Object-Oriented Approach with UML*, 5th ed. Wiley, 2015.